LAPORAN PRAKTIKUM PEMROGRAMAN WEB

MODUL 6

[react]



Disusun Oleh :

Yensen / 103122430054

KELAS

SE-08-02

PROGRAM STUDI S1 SOFTWARE ENGINEERING FAKULTAS INFORMATIKA

TELKOM UNIVERSITY PURWOKERTO 2026

Laporan Praktikum Pemrograman Web

Modul 06 – React Dasar: Aplikasi Manajemen Peserta Praktikum

1. Tujuan

1. Memahami cara membangun antarmuka web dengan React menggunakan JSX dan functional component.
2. Memahami pengiriman data dan fungsi antar komponen menggunakan props.
3. Memahami pengelolaan data yang berubah menggunakan state (`useState`) dan event handler.
4. Memahami cara menampilkan daftar data dengan `map()` dan `key` yang stabil.
5. Membangun satu aplikasi utuh yang dapat menambah, mengubah, menghapus, mencari, dan menyaring data peserta serta mengubah status kehadiran.

2. Dasar Teori

React adalah pustaka JavaScript untuk membangun antarmuka pengguna berbasis komponen. Tampilan dipecah menjadi bagian-bagian kecil yang dapat dipakai ulang, lalu disusun menjadi aplikasi utuh.

JSX adalah sintaks yang memungkinkan penulisan struktur tampilan yang mirip HTML di dalam kode JavaScript. JSX kemudian diubah menjadi pemanggilan fungsi JavaScript oleh alat build.

Functional component adalah fungsi JavaScript yang menerima props dan mengembalikan JSX. Nama komponen diawali huruf kapital, misalnya `FormPeserta`.

Props adalah data atau fungsi yang dikirim dari komponen induk ke komponen anak. Props bersifat *read-only*: komponen anak tidak boleh mengubahnya langsung. Jika anak perlu mengubah data, induk mengirim fungsi penanganan event melalui props.

State adalah data yang dapat berubah selama aplikasi berjalan. Hook `useState` mengembalikan nilai state dan fungsi setter. Setiap pemanggilan setter membuat React merender ulang komponen sehingga tampilan selalu sesuai data terbaru. Array dan object harus diperbarui dengan membuat salinan baru (misalnya dengan `map`, `filter`, atau operator spread), bukan dimutasi langsung.

Controlled input adalah input yang nilainya dikendalikan state melalui atribut `value` dan `onChange`, sehingga state menjadi satu-satunya sumber kebenaran bagi isi form.

Render list dan key. Daftar ditampilkan dengan `map()`. Setiap elemen hasil `map()` wajib memiliki `key` yang unik dan stabil (pada praktikum ini memakai ID peserta) agar React dapat melacak elemen dengan benar saat data berubah.

Vite adalah alat build dan *development server* yang cepat untuk proyek web modern, termasuk proyek React.

3. Alat dan Bahan

No	Alat / Bahan	Keterangan
1	Komputer / laptop	Sistem operasi bebas
2	Node.js dan npm	Menjalankan Vite dan mengelola paket
3	Visual Studio Code	Editor kode
4	Browser (Chrome / Edge / Firefox)	Menjalankan dan menguji aplikasi
5	React 18 dan Vite	Pustaka dan alat build

4. Deskripsi Aplikasi

Aplikasi mengelola data peserta praktikum dalam satu halaman. Seluruh fitur merupakan satu kesatuan program, dan data disimpan di state selama aplikasi berjalan (reload halaman mengembalikan data awal).

4.1 Data awal

ID	Nama	NIM	Kelas	Kehadiran
m-1	Yensen	12345678	SE-08-02	Hadir (`true`)
m-2	Cia	87654321	SE-08-01	Belum hadir (`false`)
m-3	Given	13246578	SE-08-02	Belum hadir (`false`)

ID dan NIM disimpan sebagai string. NIM berupa string agar angka nol di depan tidak hilang (contoh `00123001`).

4.2 Fitur

1. **Tambah peserta** – peserta baru ditambahkan di akhir array dengan ID baru dan status belum hadir.
2. **Ubah peserta** – tombol Edit mengisi form; tombol utama berubah menjadi *Simpan perubahan* dan muncul tombol *Batal*.
3. **Hapus peserta** – berdasarkan ID; bila peserta sedang diedit, mode edit dibatalkan.
4. **Ubah kehadiran** – tombol *Tandai hadir* / *Batalkan hadir* per peserta.
5. **Pencarian** – sebagian nama atau NIM, tidak peka huruf besar/kecil, spasi awal dan akhir diabaikan.
6. **Filter kelas** – Semua kelas, SE-08-01, atau SE-08-02; dapat dikombinasikan dengan pencarian.
7. **Ringkasan** – Total peserta, Hadir, Belum hadir (dari seluruh data) dan Hasil ditampilkan (dari daftar yang terlihat).
8. **Kondisi kosong** – pesan *Belum ada peserta.* atau *Peserta tidak ditemukan.*

4.3 Aturan validasi

Input	Aturan	Pesan jika salah
Nama	Wajib diisi, minimal 3 karakter setelah spasi awal/akhir dirapikan	Nama wajib diisi. / Nama minimal 3 karakter.
NIM	Wajib, tepat 8 digit angka	NIM wajib diisi. / NIM harus terdiri atas tepat 8 digit angka.
NIM	Tidak boleh sama dengan NIM peserta lain (NIM milik peserta yang sedang diedit boleh tetap dipakai)	NIM sudah digunakan peserta lain.
Kelas	Harus SE-08-01 atau SE-08-02 ("Pilih kelas" tidak valid)	Kelas wajib dipilih (SE-08-01 atau SE-08-02).

Jika validasi gagal, data peserta dan isi form tidak berubah.

5. Perancangan

5.1 Pembagian komponen

Komponen	Tanggung jawab
App	Menyimpan state utama, mengolah data, dan menghubungkan seluruh komponen
`RingkasanPeserta`	Menampilkan empat angka ringkasan dari data yang diterima
`FormPeserta`	Menampilkan input dan tombol tambah, simpan, serta batal
`FilterPeserta`	Menampilkan input pencarian dan pilihan filter kelas
`DaftarPeserta`	Merender hasil pencarian/filter dengan `map()` dan menampilkan kondisi kosong
`ItemPeserta`	Menampilkan satu peserta beserta tombol Edit, Hapus, dan kehadiran

5.2 State pada komponen App

State	Fungsi
`peserta`	Array data utama peserta
`nextId`	Penghitung untuk membuat ID baru yang tidak pernah bentrok, meskipun ada peserta yang dihapus
`form`	Nilai input nama, NIM, dan kelas (controlled input)
`editId`	ID peserta yang sedang diedit; `null` berarti mode tambah
`keyword`	Teks pencarian

State	Fungsi
filterKelas	Pilihan filter kelas (semua, SE-08-01, SE-08-02)
galat	Pesan validasi per input
pesan	Pesan keberhasilan setelah tambah atau ubah

Daftar yang terlihat (hasil) tidak disimpan sebagai state, melainkan dihitung dari peserta, keyword, dan filterKelas setiap render. Dengan demikian pencarian dan filter hanya mengubah tampilan dan tidak pernah menghapus data sumber.

5.3 Struktur proyek

```
TUGAS_MOD6/
├── index.html
├── package.json
├── package-lock.json
├── vite.config.js
├── README.md
└── src/
    ├── main.jsx
    ├── App.jsx
    ├── styles.css
    └── components/
        ├── RingkasanPeserta.jsx
        ├── FormPeserta.jsx
        ├── FilterPeserta.jsx
        ├── DaftarPeserta.jsx
        └── ItemPeserta.jsx
```

6. Implementasi

6.1 Langkah kerja

1. Membuat proyek React + JavaScript dengan Vite, lalu memasang dependensi dengan npm install.
2. Menyiapkan folder src/components dan membuat enam komponen sesuai pembagian tanggung jawab.
3. Menulis seluruh state dan fungsi pengolah data pada App.jsx.
4. Mengirim data dan fungsi penanganan event ke komponen anak melalui props.
5. Menambahkan gaya tampilan pada styles.css dengan tema warna Telkom University (merah, putih, abu gelap).
6. Menjalankan aplikasi dengan npm run dev, menguji seluruh skenario, lalu memastikan npm run build berhasil.

Perintah menjalankan:

```
npm install
npm run dev
```

6.2 Kode program

src/App.jsx

```
import { useState } from "react";
import RingkasanPeserta from "./components/RingkasanPeserta.jsx";
import FormPeserta from "./components/FormPeserta.jsx";
import FilterPeserta from "./components/FilterPeserta.jsx";
import DaftarPeserta from "./components/DaftarPeserta.jsx";

const DATA_AWAL = [
  { id: "m-1", nama: "Yensen", nim: "12345678", kelas: "SE-08-02", hadir: true },
  { id: "m-2", nama: "Cia", nim: "87654321", kelas: "SE-08-01", hadir: false },
  { id: "m-3", nama: "Given", nim: "13246578", kelas: "SE-08-02", hadir: false },
];

const FORM_KOSONG = { nama: "", nim: "", kelas: "" };
const KELAS_VALID = ["SE-08-01", "SE-08-02"];

function App() {
  const [peserta, setPeserta] = useState(DATA_AWAL);
  const [nextId, setNextId] = useState(4);
  const [form, setForm] = useState(FORM_KOSONG);
  const [editId, setEditId] = useState(null);
  const [keyword, setKeyword] = useState("");
  const [filterKelas, setFilterKelas] = useState("semua");
  const [galat, setGalat] = useState({});
  const [pesan, setPesan] = useState("");

  // Daftar yang terlihat: dihitung dari data utama, pencarian, dan filter
  const kataCari = keyword.trim().toLowerCase();
  const hasil = peserta.filter((p) => {
    const cocokCari =
      kataCari === "" ||
      p.nama.toLowerCase().includes(kataCari) ||
      p.nim.toLowerCase().includes(kataCari);
    const cocokKelas = filterKelas === "semua" || p.kelas === filterKelas;
    return cocokCari && cocokKelas;
  });

  function resetForm() {
    setForm(FORM_KOSONG);
    setEditId(null);
    setGalat({});
  }

  function handleFormChange(e) {
    const { name, value } = e.target;
    setForm((lama) => ({ ...lama, [name]: value }));
    setGalat((lama) => ({ ...lama, [name]: undefined }));
  }

  function validasi(nama, nim, kelas) {
    const hasilGalat = {};
    if (nama === "") {
      hasilGalat.nama = "Nama wajib diisi.";
    } else if (nama.length < 3) {
      hasilGalat.nama = "Nama minimal 3 karakter.";
    }
    if (nim === "") {
```

```
    hasilGalat.nim = "NIM wajib diisi.";
  } else if (!/^\d{8}$/.test(nim)) {
    hasilGalat.nim = "NIM harus terdiri atas tepat 8 digit angka.";
  } else if (peserta.some((p) => p.nim === nim && p.id !== editId)) {
    hasilGalat.nim = "NIM sudah digunakan peserta lain.";
  }
  if (!KELAS_VALID.includes(kelas)) {
    hasilGalat.kelas = "Kelas wajib dipilih (SE-08-01 atau SE-08-02).";
  }
  return hasilGalat;
}

function handleSubmit(e) {
  e.preventDefault();
  const nama = form.nama.trim();
  const nim = form.nim.trim();
  const kelas = form.kelas;

  const hasilGalat = validasi(nama, nim, kelas);
  if (Object.keys(hasilGalat).length > 0) {
    setGalat(hasilGalat);
    setPesan("");
    return; // data dan isi form tidak berubah
  }

  if (editId === null) {
    const baru = { id: `m-${nextId}`, nama, nim, kelas, hadir: false };
    setPeserta([...peserta, baru]);
    setNextId(nextId + 1);
    setPesan(`Peserta ${nama} berhasil ditambahkan.`);
  } else {
    setPeserta(
      peserta.map((p) => (p.id === editId ? { ...p, nama, nim, kelas } : p))
    );
    setPesan(`Data peserta ${nama} berhasil diperbarui.`);
  }
  resetForm();
}

function handleEdit(id) {
  const target = peserta.find((p) => p.id === id);
  if (!target) return;
  setEditId(id);
  setForm({ nama: target.nama, nim: target.nim, kelas: target.kelas });
  setGalat({});
  setPesan("");
}

function handleBatal() {
  resetForm();
  setPesan("");
}

function handleHapus(id) {
  setPeserta(peserta.filter((p) => p.id !== id));
  if (editId === id) {
```

```
    resetForm();
  }
  setPesan("");
 }

 function handleToggleHadir(id) {
  setPeserta(peserta.map((p) => (p.id === id ? { ...p, hadir: !p.hadir } : p)));
  setPesan("");
 }

 return (
  <main className="app">
   <h1>Manajemen Peserta Praktikum</h1>

   <RingkasanPeserta peserta={peserta} jumlahTampil={hasil.length} />

   <section className="panel">
    <FormPeserta
     form={form}
     galat={galat}
     modeEdit={editId !== null}
     onChange={handleFormChange}
     onSubmit={handleSubmit}
     onBatal={handleBatal}
    />
    {pesan && (
     <p className="pesan-sukses" role="status">
      {pesan}
     </p>
    )}
    <FilterPeserta
     keyword={keyword}
     filterKelas={filterKelas}
     onKeywordChange={setKeyword}
     onFilterChange={setFilterKelas}
    />
   </section>

   <DaftarPeserta
    daftar={hasil}
    totalData={peserta.length}
    editId={editId}
    onEdit={handleEdit}
    onHapus={handleHapus}
    onToggleHadir={handleToggleHadir}
   />
  </main>
 );
}

export default App;
```

src/components/RingkasanPeserta.jsx

```
function RingkasanPeserta({ peserta, jumlahTampil }) {
  const total = peserta.length;
```

```
  const hadir = peserta.filter((p) => p.hadir).length;
  const belumHadir = total - hadir;

  const item = [
    { label: "Total peserta", nilai: total },
    { label: "Hadir", nilai: hadir },
    { label: "Belum hadir", nilai: belumHadir },
    { label: "Hasil ditampilkan", nilai: jumlahTampil },
  ];

  return (
    <section className="ringkasan" aria-label="Ringkasan peserta">
      {item.map((i) => (
        <div className="ringkasan-kartu" key={i.label}>
          <span className="ringkasan-label">{i.label}</span>
          <strong className="ringkasan-nilai">{i.nilai}</strong>
        </div>
      ))}
    </section>
  );
}

export default RingkasanPeserta;
```

src/components/FormPeserta.jsx

```
function FormPeserta({ form, galat, modeEdit, onChange, onSubmit, onBatal }) {
  return (
    <form className="form" onSubmit={onSubmit} noValidate>
      <h2>{modeEdit ? "Ubah peserta" : "Tambah peserta"}</h2>

      <div className="form-grup">
        <label htmlFor="nama">Nama mahasiswa</label>
        <input id="nama" name="nama" type="text" value={form.nama}
          onChange={onChange} placeholder="Contoh: Yensen" autoComplete="off" />
        {galat.nama && <span className="galat" role="alert">{galat.nama}</span>}
      </div>

      <div className="form-grup">
        <label htmlFor="nim">NIM</label>
        <input id="nim" name="nim" type="text" inputMode="numeric" value={form.nim}
          onChange={onChange} placeholder="8 digit angka" autoComplete="off" />
        {galat.nim && <span className="galat" role="alert">{galat.nim}</span>}
      </div>

      <div className="form-grup">
        <label htmlFor="kelas">Kelas</label>
        <select id="kelas" name="kelas" value={form.kelas} onChange={onChange}>
          <option value="">Pilih kelas</option>
          <option value="SE-08-01">SE-08-01</option>
          <option value="SE-08-02">SE-08-02</option>
        </select>
        {galat.kelas && <span className="galat" role="alert">{galat.kelas}</span>}
```

```
    </div>

    <div className="form-aksi">
      <button type="submit" className="btn btn-utama">
        {modeEdit ? "Simpan perubahan" : "Tambah peserta"}
      </button>
      {modeEdit && (
        <button type="button" className="btn" onClick={onBatal}>Batal</button>
      )}
    </div>
  </form>
 );
}

export default FormPeserta;
```

src/components/FilterPeserta.jsx

```
function FilterPeserta({ keyword, filterKelas, onKeywordChange, onFilterChange }) {
 return (
  <div className="filter">
   <h2>Cari dan saring</h2>
   <div className="filter-baris">
    <div className="form-grup">
     <label htmlFor="cari">Cari nama atau NIM</label>
     <input id="cari" type="text" value={keyword}
      onChange={(e) => onKeywordChange(e.target.value)}
      placeholder="Ketik nama atau NIM" autoComplete="off" />
    </div>
    <div className="form-grup">
     <label htmlFor="filter-kelas">Filter kelas</label>
     <select id="filter-kelas" value={filterKelas}
      onChange={(e) => onFilterChange(e.target.value)}>
      <option value="semua">Semua kelas</option>
      <option value="SE-08-01">SE-08-01</option>
      <option value="SE-08-02">SE-08-02</option>
     </select>
    </div>
   </div>
  </div>
 );
}

export default FilterPeserta;
```

src/components/DaftarPeserta.jsx

```
import ItemPeserta from "./ItemPeserta.jsx";

function DaftarPeserta({ daftar, totalData, editId, onEdit, onHapus, onToggleHadir }) {
 if (totalData === 0) {
  return <p className="kosong">Belum ada peserta.</p>;
 }
 if (daftar.length === 0) {
  return <p className="kosong">Peserta tidak ditemukan.</p>;
 }
```

```
 return (
  <ul className="daftar">
    {daftar.map((p) => (
      <ItemPeserta
        key={p.id}
        peserta={p}
        sedangDiedit={p.id === editId}
        onEdit={onEdit}
        onHapus={onHapus}
        onToggleHadir={onToggleHadir}
      />
    ))}
  </ul>
 );
}

export default DaftarPeserta;
```

src/components/ItemPeserta.jsx

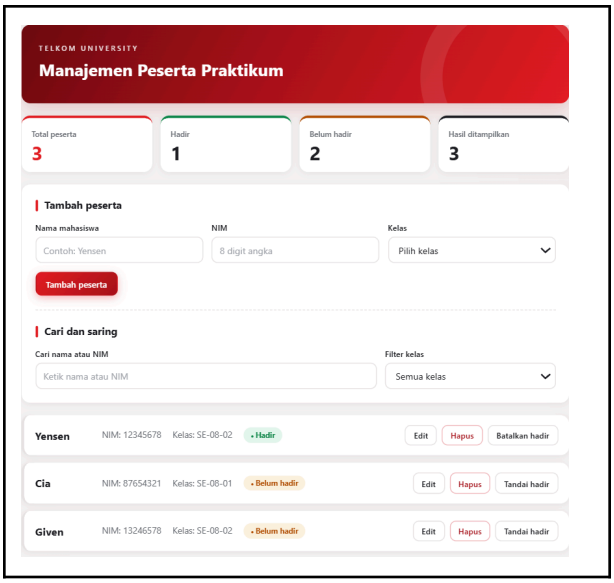
```
function ItemPeserta({ peserta, sedangDiedit, onEdit, onHapus, onToggleHadir }) {
 const { id, nama, nim, kelas, hadir } = peserta;

 return (
  <li className={"item" + (sedangDiedit ? " item-diedit" : "")}>
    <div className="item-info">
      <strong className="item-nama">{nama}</strong>
      <span>NIM: {nim}</span>
      <span>Kelas: {kelas}</span>
      <span className={"status " + (hadir ? "status-hadir" : "status-belum")}>
        {hadir ? "Hadir" : "Belum hadir"}
      </span>
    </div>
    <div className="item-aksi">
      <button type="button" className="btn" onClick={() => onEdit(id)}>Edit</button>
      <button type="button" className="btn btn-bahaya" onClick={() => onHapus(id)}>Hapus</button>
      <button type="button" className="btn" onClick={() => onToggleHadir(id)}>
        {hadir ? "Batalkan hadir" : "Tandai hadir"}
      </button>
    </div>
  </li>
 );
}

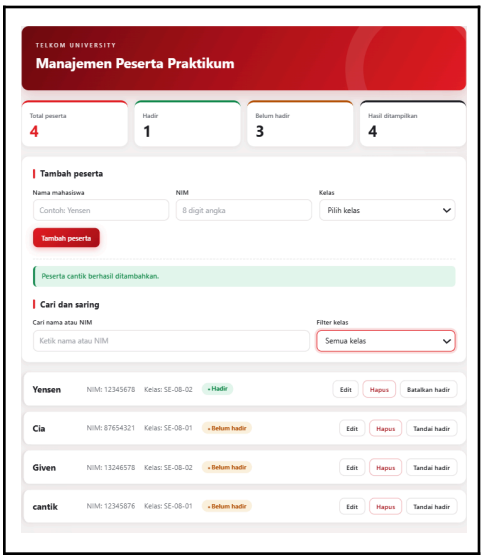
export default ItemPeserta;
```

7. Hasil dan Pengujian

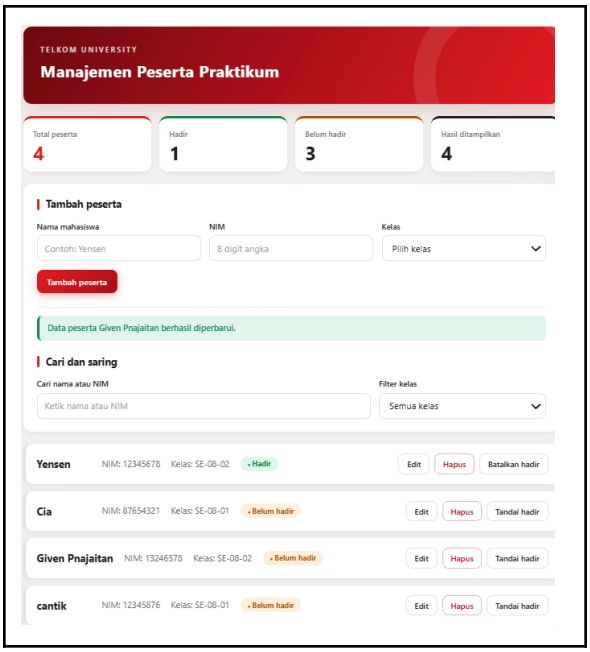
Gambar 1. Kondisi awal aplikasi (3 peserta; total 3, hadir 1, belum hadir 2, hasil ditampilkan 3)



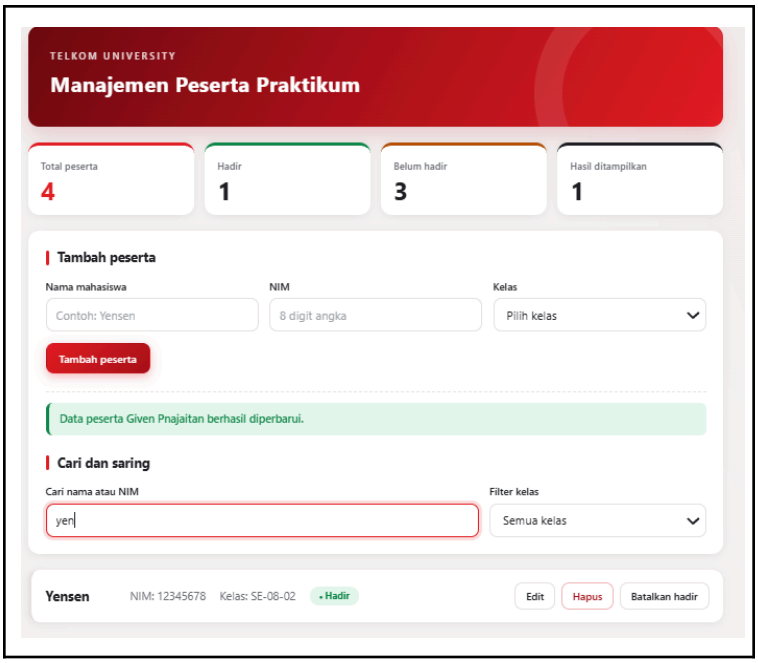
Gambar 2. Hasil tambah peserta



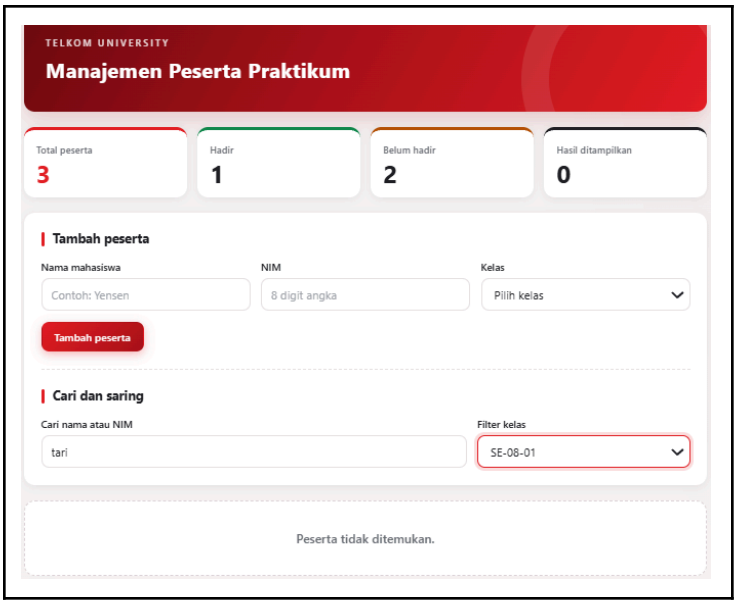
Gambar 3. Hasil edit peserta



Gambar 4. Hasil pencarian " Yen "



Gambar 5. Kondisi "Peserta tidak ditemukan



8. Pembahasan

Pembagian komponen dan aliran data. Seluruh state disimpan di komponen App, sedangkan komponen anak hanya menerima data dan fungsi melalui props. Komponen anak tidak mengubah props secara langsung; setiap perubahan dilakukan dengan memanggil fungsi dari App (misalnya onEdit, onHapus, onToggleHadir), yang kemudian memperbarui state melalui setter. Pola ini membuat data hanya mengalir satu arah (dari induk ke anak) sehingga perilaku aplikasi mudah ditelusuri.

Controlled input dan validasi. Ketiga input form dikendalikan state form. Saat disubmit, nama dan NIM dirapikan dengan trim() lalu divalidasi. Bila ada kesalahan, fungsi berhenti sebelum mengubah data sehingga jumlah peserta dan isi form tetap. Pencocokan NIM ganda mengecualikan peserta yang sedang diedit (p.id !== editId), sehingga NIM milik sendiri tetap boleh dipakai saat mengedit. Atribut noValidate dipasang pada form agar pesan validasi yang tampil adalah pesan buatan aplikasi, bukan pesan bawaan browser.

Penanganan submit. Fungsi handleSubmit memanggil e.preventDefault() sehingga halaman tidak dimuat ulang, dan karena memakai elemen <form> dengan onSubmit, tombol Enter juga dapat mengirim form.

Pembaruan state yang aman. Data tidak pernah dimutasi langsung. Tambah memakai operator spread, ubah dan toggle kehadiran memakai map() (hanya objek dengan ID yang cocok yang diganti, sehingga ID, urutan, dan status lain tidak berubah), dan hapus memakai filter() berdasarkan ID.

ID yang stabil. ID baru dibuat dari penghitung nextId, bukan dari panjang array. Dengan cara ini, ID tidak akan bentrok walaupun ada peserta yang dihapus, dan ID dapat dipakai sebagai key yang stabil pada map().

Pencarian dan filter. Daftar hasil dihitung dari data utama setiap render, bukan disimpan sebagai state terpisah. Konsekuensinya, pencarian dan filter hanya memengaruhi tampilan; data sumber tidak berubah, dan angka Total, Hadir, serta Belum hadir tetap dihitung dari seluruh data sedangkan Hasil ditampilkan dihitung dari daftar yang terlihat. Kata pencarian dirapikan dengan trim() dan toLowerCase(), sehingga pencarian kosong atau hanya spasi tidak membatasi hasil.

Kondisi kosong. `DaftarPeserta` membedakan dua kondisi: data sumber kosong (*Belum ada peserta.*) dan data ada tetapi tidak ada yang cocok dengan pencarian/filter (*Peserta tidak ditemukan.*).

Kendala dan penyelesaian. Kendala utama adalah menjaga agar angka ringkasan selalu mengikuti data terbaru dan tidak terpengaruh pencarian. Hal ini diselesaikan dengan menghitung ringkasan langsung dari array `peserta` yang diterima sebagai props, sehingga tidak ada nilai ganda yang harus disinkronkan secara manual.

9. Kesimpulan

1. Aplikasi Manajemen Peserta Praktikum berhasil dibangun dengan React dan Vite menggunakan JSX, functional component, props, `useState`, event handler, serta render list dengan `map()` dan `key`.
2. Seluruh fitur (tambah, ubah, hapus, cari, filter, ubah kehadiran, dan ringkasan) bekerja sebagai satu kesatuan dan sesuai dengan 12 skenario pengujian.
3. Menyimpan state utama di komponen induk dan mengirimkannya melalui props membuat aliran data jelas dan komponen mudah dipakai ulang.
4. Menghitung data turunan (daftar hasil dan angka ringkasan) langsung dari state utama menjaga tampilan selalu konsisten dengan data terbaru.

Daftar Pustaka

1. React Team. *React Documentation*. https://react.dev
2. Vite. *Vite Documentation*. https://vite.dev
3. Modul Praktikum Pemrograman Web, Modul 06 – React Dasar.